

Large language model-based agent Schema and library for automated building energy analysis and modeling

Liang Zhang^{a,b,*}, Xiaoqin Fu^a, Yanfei Li^c, Jianli Chen^{d,e}

^a University of Arizona, Tucson, AZ, United States

^b National Renewable Energy Laboratory, Golden, CO, United States

^c Oak Ridge National Laboratory, Oak Ridge, TN, United States

^d Tongji University, Shanghai, China

^e University of Utah, Salt Lake City, UT, United States

ARTICLE INFO

Keywords:

Building energy analysis
Building energy modeling
Large language model
Agentic workflow
Agent schema and library

ABSTRACT

Large language models (LLMs) agents can function as autonomous, interactive, goal-oriented systems, but in the building energy sector, there is currently no structured paradigm that researchers and engineers can follow to create, access, and share effective LLM agents without starting from scratch. This paper introduces a JSON-based agent schema designed to structure the description of LLM agents. Additionally, the paper introduces an open-source library on GitHub that serves as a centralized repository for LLM agents designed for building energy analysis and modeling, all structured according to this schema. This library is publicly accessible, allowing users to utilize and upload agents, thereby enhancing the accessibility of LLM agents. The case studies demonstrate the schema's effectiveness with four example agents developed across different platform. These applications, developed on diverse platforms, successfully execute and seamlessly align with the proposed schema and can be reproduced without additional information beyond the schema.

1. Introduction

Buildings consume 40 % of primary energy and are responsible for over one third of carbon emission globally [1]. Building energy analysis and modeling employ physics-based energy simulation like EnergyPlus [2] and data-driven method including data analysis and machine learning to evaluate and optimize energy consumption, system performance, and environmental impact. Conducting building energy analysis and modeling at the scales of individual and multiple buildings [3], as well as regional and national levels [4], is essential to understand current and future pattern of energy use [5], identifying energy-saving opportunities and enhancing energy efficiency [6], promoting sustainability [7], supporting policy development [8], reducing carbon emissions [9], and optimizing resource allocation [10].

Recently, there have been some advancements related to building energy analysis and modeling. For example, by incorporating physics-based equations into neural network models, Michalakopoulos et al. [11] achieved promising results in predicting energy consumption, paving the way for automated, data-driven energy performance assessments. Meanwhile, Lee et al. [12] introduced a metadata schema

designed to enhance the integration and functionality of virtual building models within digital twin environments, which was implemented using GPT-based applications to facilitate more efficient data management and analysis in building operations. And Song et al. [13] discussed the implementation of multizone airflow simulation using ontology-assisted GPT-based building performance simulation and assessment.

Building energy analysis and modeling, whether data-driven [14] or physics-based [15], are increasingly essential yet demand specialized knowledge in fields such as building science, thermal science, data science, energy systems engineering, and regulatory frameworks. Expanding access to these analytical tools to a broader audience is crucial for promoting energy efficiency, spurring innovation, enabling informed decision-making, and supporting sustainable practices across different sectors. Democratizing these tools can empower individuals and organizations to better understand and manage their energy usage, minimize environmental impacts, and contribute to global efforts to combat climate change.

A large language model (LLM) is a type of artificial intelligence model based on the transformer architecture, which leverages self-attention mechanisms to efficiently process and generate human

* Corresponding author at: University of Arizona, Tucson, AZ, United States.
E-mail address: liangzhang1@arizona.edu (L. Zhang).

<https://doi.org/10.1016/j.autcon.2025.106244>

Received 8 January 2025; Received in revised form 23 April 2025; Accepted 28 April 2025

Available online 5 May 2025

0926-5805/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

language by learning patterns from large-scale text datasets [16]. These models are designed to handle tasks like text generation, translation, summarization, and answering questions. For example, GPT-4 is an LLM that can generate human-like text and assist with various natural language tasks, such as writing, coding, and conversation. LLMs hold significant potential to revolutionize building energy analysis and modeling. Defined as deep neural networks with billions of parameters, transformer structure, self-attention mechanism [16], and capability of processing and generating human-like text, LLM excels in reasoning, planning, and code generation. LLMs are highly promising for automating complex data analysis and modeling tasks in the building energy sector. Leveraging natural language interfaces, LLMs enable people to analyze and optimize energy systems more effectively and efficiently. By introducing a paradigm shift, LLMs can handle fuzzy, incomplete, or ambiguous external information while smartly and reasonably managing complex and dynamic tasks. For instance, building operators can access building databases to create customized visualizations and analyses without needing to learn SQL or Python; energy modelers can save significant time by avoiding manual data entry for tools like EnergyPlus; policymakers can quickly summarize extensive reports; building designers can more effectively integrate energy efficiency into designs; and consultants can produce comprehensive energy audits with minimal effort.

LLM research has recently advanced rapidly in the field of building energy analysis and modeling. Sharma et al. [17] introduced SQLMorpher, an LLM-based framework for automating data transformation in the building sector, addressing challenges like limited domain knowledge interfaces, high training data overheads, and schema complexity, achieving 96 % accuracy on 105 real-world building energy data transformation problems. Jiang et al. [18] proposed Eplus-LLM, a fine-tuned LLM framework that automates building energy modeling by translating natural language descriptions into accurate EnergyPlus models, reducing over 95 % of modeling efforts and achieving 100 % accuracy, that the EnergyPlus engine successfully invoked all eighty instances generated by Eplus-LLM, although the authors did not specify which data has been used; thereby enabling scalable solutions for tasks like urban building energy modeling. Zhang and Chen [19] introduced an innovative interpretable machine learning framework combining Shapley values with the in-context learning capabilities of LLMs to enhance transparency and trust in machine learning control for HVAC systems, demonstrated through a case study on model predictive control-based precooling during demand response events. Khadka and Zhang [20] proposed a methodology leveraging LLMs to address scalability challenges in data-driven building energy modeling by automating data handling and modeling processes, achieving high code generation accuracy and significantly reducing human effort through a bi-sequential prompting framework. Forth and Borrmann [21] proposed a methodology leveraging semantic textual similarity and fine-tuned LLMs to automate the enrichment of missing information in building information models, achieving high accuracy in matching semantic data and demonstrating minimal deviation in Building Energy Performance Simulation results compared to manually enriched models.

However, the key limitation of these approaches is that they lack structured frameworks and formats for agent workflow definition, shared schemas, or structured ways to scale LLM-based automation beyond the experimental phase, making their work hard to replicate. Meanwhile, they integrate LLMs with specific tools rather than defining a tool-agnostic methodology. This makes it challenging for others to use or integrate these workflows into their own. On the contrary, this paper introduces an initial exploration of a platform-agnostic library and a tool-independent schema, which were designed to streamline the

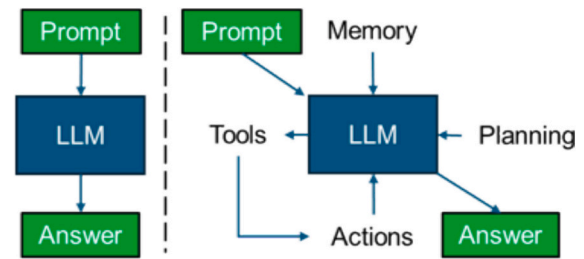


Fig. 1. LLM prompt engineering (left) vs. LLM agent (right) [28].

description of LLM agents, enhancing reusability, scalability, and collaboration.

While LLMs hold great promise, their development faces significant challenges. Prompting, the basic interaction of asking questions and receiving answers (Fig. 1, left), is straightforward but limited. Prompt engineering, which involves carefully crafting and refining these prompts, allows for handling more complex tasks. However, even prompt engineering has its limits as task complexity increases [22]. LLM agents [23] (Fig. 1, right) expand the capabilities of prompting by functioning as an autonomous, goal-oriented system with an LLM at its core for reasoning. The essential element of the LLM agent is the Planning module, which deconstructs intricate tasks and enhances performance through self-reflection. It employs various modular Tools, such as specialized functions and software (e.g., EnergyPlus and Python IDE), to extend its capabilities, carries out internal or external Actions based on these tools, and uses both short-term and long-term Memory to improve functionality and maintain contextual knowledge. The agentic LLM workflow refers to a process that incorporates LLM agent(s), whose design pattern includes reflection (self-assessment) [24], tool use [25], planning [26], and multi-agent collaboration [27]. LLM agentic workflow significantly enhances the potential of LLMs, enabling decentralized, coordinated decision-making across various components of complex systems including building energy systems. Within this workflow, LLM agents collaborate among themselves and with non-LLM components, such as rule-based and physics-based tools and systems in buildings, thereby facilitating coordinated efforts to manage complex engineering processes in buildings.

LLM agents have only recently begun to be explored in the field of building energy analysis and modeling. Xiao and Xu [29] proposed an LLM-based multi-agent framework based on CrewAI for automating building energy optimization tasks, demonstrating its effectiveness in processing building information, diagnosing performance, and recommending retrofits, while significantly reducing repetitive labor and costs. Zhang et al. [28,30] demonstrated an LLM agentic workflow for automating physics-based building energy modeling tool, using four specialized agents to transform building descriptions into error-free EnergyPlus models with greater accuracy, reliability, and efficiency than existing methods.

Although LLM agents offer an advanced paradigm for utilizing LLMs, their development involves complexities beyond simple prompt engineering. Prompt engineering only requires crafting a prompt and pairing it with an LLM foundation model, while developing effective LLM agents demands a sophisticated, multifaceted approach. This process often includes defining multiple agents and their planning, action, tool, and memory. Unlike prompt engineering, where sharing successful implementations may simply involve distributing prompt and model details, sharing LLM agents requires a more complex setup, including detailed

workflows and architectures. Various agent paradigms have been developed to enable complex agentic workflows. One such example is the Reasoning and Acting (ReAct) framework, in which an LLM alternates between generating intermediate reasoning steps and performing actions—such as retrieving information or invoking tools—while iteratively interacting with its environment until the task is completed [31]. Sequential/flow engineering systematically organizes tasks and agents to achieve specific outcomes, while multi-agent systems use multiple LLM agents to address different problem components, often coordinated by a supervisory agent. While a few LLM agent libraries exist, such as LangChain's official example agent library (available at <https://github.com/langchain-ai/langgraph/tree/main/examples>), the lack of a structured framework for creating, accessing, and sharing LLM agents often compels researchers and engineers to start from scratch. This represents a significant obstacle to their widespread adoption and collaborative development.

Moreover, numerous agent development platforms, environments, and tools, including AutoGen, CrewAI, LangChain ecosystem, and many others, offer a range of features tailored to different aspects of agent development, enabling developers to select options that best meet their needs. In particular, AutoGen is a Python-based framework designed to enable autonomous agent collaboration. It allows agents to communicate dynamically, exchange information, and adapt based on real-time task demands. Key features include agent collaboration, code-generation integration, and support for tasks involving problem-solving, creative collaboration, or automation requiring multi-agent feedback loops [32]. LangGraph is a framework for building agentic and multi-agent applications. It can be easily integrated with other agent frameworks, allowing for the creation of multi-agent systems where individual agents are built with different frameworks. LangGraph offers features like persistence, streaming, and memory management, which are essential for complex workflows [33]. In addition, CrewAI is a high-level framework that prioritizes rapid prototyping and developer experience. It provides a streamlined development experience, making it particularly accessible for those new to agentic AI development. CrewAI offers easy-to-use workflow management, quick setup, and iteration, enabling developers to get started quickly with intuitive project structures [34] [35].

However, the agent development landscape is highly fragmented, with each platform providing unique functionalities that are not readily interchangeable. This fragmentation poses significant challenges for transitioning between tools or integrating them into complex workflows. Additionally, the absence of a structured framework for developing, sharing, and deploying LLM agents exacerbates these issues, impeding collaboration and slowing the adoption of LLM agents in specialized fields including building energy analysis and modeling.

To address these gaps, this paper proposes the development of a comprehensive agent library that transcends the limitations of specific platforms. This library will be built on a structured, platform-agnostic schema that focuses solely on the agentic workflow and procedural aspects of LLM agents, independent of the underlying tools. By providing detailed workflows and structured definitions within this library, developers will be able to share and collaborate more effectively, fostering a more unified and versatile approach to LLM agent development. This proposed schema and library aim to facilitate the systematic reuse and deployment of LLM agents, particularly in the building energy sector. The establishment of such a library will serve as a foundational step toward creating a more cohesive and accessible environment for the development and application of LLM agents, ultimately advancing the field of building energy analysis and modeling.

The proposed schema and library for LLM agents offer several

advantages over prior approaches in building energy analysis and modeling. Existing methods often face challenges such as high computational demands, lack of standardization, and limited flexibility. Integrating LLMs into building energy systems can lead to increased energy use and environmental impact due to the substantial computational resources required. This issue is particularly pronounced in multimodal models and systems with multiple specialized agents, which can exacerbate energy consumption [36]. Many existing LLM agent frameworks are platform-specific, lacking a structured framework for developing, sharing, and deploying LLM agents. This fragmentation impedes collaboration and slows the adoption of LLM agents in specialized fields, including building energy analysis and modeling. Some frameworks may not offer the flexibility needed to optimize for specific use cases, potentially requiring more development time and leading to the reinvention of existing solutions. The absence of structured definitions and workflows can hinder effective sharing and collaboration among developers [37].

Instead, the proposed schema offers a structured framework that is not tied to specific platforms, enabling seamless integration and collaboration across diverse tools and systems. This design enhances flexibility and adaptability, allowing developers to transition between different platforms or integrate them into complex workflows more efficiently. By concentrating on the agentic workflow and procedural elements of LLM agents, the schema provides a clear and structured approach to agent development. This focus simplifies the development process, making it more accessible and efficient for developers. The library supports the systematic reuse and deployment of LLM agents, particularly in the building energy sector. This capability streamlines the application of LLM agents in real-world scenarios, enhancing their practicality and impact. In summary, our approach addresses the limitations of prior approaches by providing a structured, flexible, and efficient framework for LLM agent development, fostering collaboration, and enhancing the applicability of LLM agents in building energy analysis and modeling.

The paper is structured as follows. The methodology section details the schema definition for LLM agentic workflows and the design structure of the agent library. The case study section demonstrates the application of the developed schema within the library through four examples. The results and discussion section provides evaluation and discussion of the case study results, especially the agent performance. The conclusion section summarizes the findings, limitations, and future work.

2. Methodology

The Methodology section is organized into two core components: 1) the schema design for LLM agentic workflows, and 2) the structural design of the agent library. First, the schema is designed around essential elements such as planning, action, memory, and tools, supporting both individual and coordinated multi-agent workflows.

The proposed schema is designed to support a wide range of building energy analysis tasks, but only if these tasks can be summarized using the core structure of “planning, action, tools, and memory”. As shown in Fig. 2, the paper employs a structure inspired by, but distinct from, LangGraph, a leading agent workflow tool, which integrates the following key components: 1) START/END point indicating the start and end of the workflow. 2) nodes, including agent nodes that call and get output from LLMs to plan, reason, and generate content, and action nodes that call, execute, and get output from tools. 3) tools, which can be called by action nodes to conduct detailed and domain specific tasks. 4) edges, which connect two or multiple nodes. Simple edges link two

Example: Agent for getting weather data for buildings

Agent Node:

- *Question:* Provide your building location, whether in zip code or address, and ask for any weather-related data
- *Thought:* based on user's input and the input from action node, call the weather-API tool, or end the agent
- *Action:* call the weather-API-tool
- *Observation:* whether the user's data request is done

Conditional Edge: If question is answered, call the action node; else END

Tool: weather-API tool

Action Node: Call the weather-API tool, and send the output of the tool to Agent Node

Memory: keep all history messages of all nodes

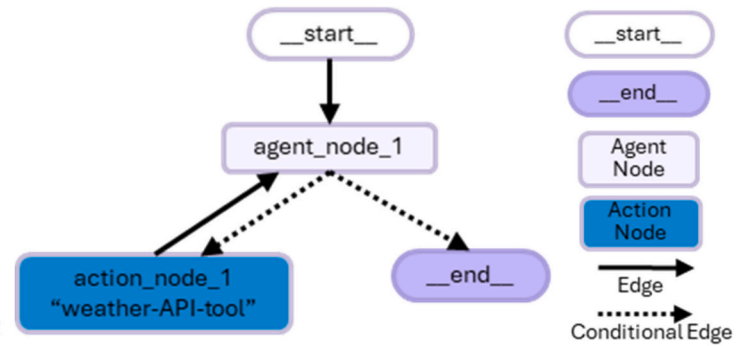


Fig. 2. Diagram of schema design for LLM.

Table 1

Dictionary structure of the developed JSON-based schema.

No.	Keys	Values	Description
1	id	String	A unique identifier for the LLM agentic workflow.
2	description	String	A description of the workflow, including its goal, input example, and expected output. This is not a system message but just a description.
3	model	String	The LLM foundation model that drives the entire LLM agentic workflow, e.g., "gpt-4o."
4	nodes	A list of dictionaries	A list of dictionaries. Each dictionary describes one agent node, which serves as the main entities forming the workflow.
5	edges	A list of dictionaries	A list of dictionaries. Each dictionary describes one edge. Each dictionary describes one edge.
6	tools	A list of dictionaries	A list of tools that can be invoked within this agentic workflow. Each dictionary describes one tool.
7	long_term_memory	A list of strings	A list of strings that defines the long-term memory of this agentic workflow. If set to ["none"], no long-term memory will be applied to this workflow.

components, such as the START point, agent node, action node, or END point. Conditional edges create more complex connections among multiple nodes—one serving as the input node and the others as output nodes—with the path determined by the input node's evaluation and decision. 5) memories, which details what input and output message/data are transmitted between nodes during the agent run. These elements work together to define the flow and logic of the agent operations.

As demonstrated in the introduction, the paper aims to use a structured approach to ensure the workflow achieves its intended goal by defining components and their connections. The paper developed a JSON-based and dictionary-formatted schema shown in Table 1 to accomplish this goal. The JSON dictionary contains the following key-

value pairs: 1) id (string): This identifies the entire workflow, e.g., "get_building_weather_data"; 2) description (string): A description of the workflow, including its goal, input example, and expected output. This is not a system message but just a description; 3) model (string): The LLM foundation model that drives the entire LLM agentic workflow, e.g., "gpt-4o;" 4) nodes (a list of dictionaries): A list of dictionaries including all agent node and action node dictionary, which serve as the main entities forming the workflow. 5) edges (a list of dictionaries): A list of dictionaries including all simple edge dictionary and conditional dictionary; 6) tools (a list of dictionaries): A list of tools that can be invoked within this agentic workflow. Each tool is described in the form of a JSON dictionary; 7) long_term_memory (a list of strings): a list of strings

Table 2

Sub-dictionary details.

Dictionary	Keys	Values	Description
Node Dictionary	id	String	A unique ID identifying the agent node.
	description	String	A description of the workflow, including its goal, input example, and expected output. This is not a system message but just a description.
	system_message	String	The key prompts to drive the planning and reasoning of the LLM agents.
	type	String	A string indicates whether this node is "agent" or "action."
	agent_tools	A list of strings	A list of strings identifies the tool pool, which is a subset of the tools defined in "tools."
	short_term_memory	A list of strings	A list of strings defines the memory context for this agent. If set to ["all"], the node receives all memory from the current workflow run.
Edge Dictionary	human_input	String	A string indicates whether the node requires human users' input, with options of "always", "never", "first_time", or "conditional."
	id	string	A unique ID for the simple edge.
	type	string	It indicates whether the edge is "simple", "conditional", or "optional."
Tool Dictionary	connection	A list of a string and a list	The string in the list defines the starting node of the edge arrow. The list in the list defines the ending node(s) of the edge arrow. If it is a simple or optional edge, the list only has one element of the agent that edge connects; if it is a conditional edge, the list lists all the agents that the conditional edge connects. The condition is defined in the starting node.
	id	String	The name of the tool, e.g., "python-code-executor" and "weather-API".
	description	String	A general description of the input and output of the tool.
	content	String	The name of the tool function or class, which can be built-in or self-defined functions/classes.

that defines the long-term memory of this agentic workflow. If set to ["none"], no long-term memory will be applied to this workflow. The node, edge, and tool dictionary are defined in the next paragraph. (See Table 2.)

For **node dictionary**, the keys are: 1) id (string); 2) description (string); 3) system_message (string): the key prompts to drive the planning and reasoning of the LLM agents; 4) type (string): Indicates whether this node is agent node or action node; 5) agent_tools (a list of strings): listed all the needed tools for this agent, which is a subset of the tools defined earlier in "tools;" 6) short_term_memory (a list of strings): define the memory context for this agent. If set to ["all"], the node receives all memory from the current workflow run. If it is set to an integer, it defines how many previous iterations of memory will be provided to the next agent; 7) human_input (string): indicate whether the node requires human users' input, with options of "always", "never", "first_time", or "conditional". For **edge dictionary**, the keys include: 1) id (string); 2) type (string): indicates whether this edge is a simple edge (a mandatory one-to-one link), a conditional edge (a one-to-n link where one of the n outcomes occurs based on a condition, $n \geq 2$), or an optional edge (a link that may or may not occur); 3) connection (a list of a string and a list): the string in the list defines the starting node of the edge arrow. The list in the list defines the ending node(s) of the edge arrow. If it is a simple or optional edge, the list only has one element of the node that edge connects; if it is a conditional edge, the list lists all the agents that the conditional edge connects. The condition is defined in the starting node of the edge. For **tool dictionary**, the keys include 1) id (string): the name of the tool, e.g., "python-code-executor" and "weather-API;" 2) description (string): a general description of the input and output of the tool; 3) content (string): the name of the tool function or class, which can be built-in or self-defined functions/classes.

Listing 1 shows a schema in JSON file as an example to define the LLM agentic workflow described in Fig. 2 about getting weather data for a specific building. This schema exactly describes the designed workflow, components, tools, and their connections shown in Fig. 2, providing a structured method, instead of a diagram and figures, to quantify an LLM agent.

The agent library, built on the schema outlined above, will be introduced shortly as a structured framework for efficiently organizing and managing agentic workflows. The library is organized as follows.

- (1) A code template folder called "**code_templates**", where multiple code templates such as "code_template_LangGraph.py/.ipynb",

"code_template_CrewAI.py/.ipynb", and "code_template_AutoGen.py/.ipynb" locate. The templates provide examples and guidance to developers to create agentic workflows under multiple agent platforms based on the JSON-based schema. The code template for LangGraph is shown in Listing 2. Placeholders and examples in the template are highlighted with comments and a grey background, indicating where nodes, edges, tools, and other components should be created, connected, and defined. The template is designed for ease of use, allowing people to seamlessly convert the JSON schema into the necessary code structure.

- (2) An agentic workflow folder named "**LLM_agentic_workflows**." Within this folder, each agentic workflow has a subfolder named according to its ID. Each subfolder contains: i) the JSON schema file based on the developed structured introduced earlier (Listing 1 as an example), ii) a run.py (Python script), run.ipynb (Jupyter Notebook), and/or cloud service like Google Colab to execute the entire workflow without errors, iii) a "tools" subfolder with files defining all the tools listed in the JSON schema if not defined in the run.py file. The files in the "tools" subfolder are usually stored in ".py" format, iv) an "evaluation" subfolder with scripts to run multiple tests under different scenarios and manually verify and evaluate their functionality, and v) a "data" subfolder that stores all input, temporary, and output files. The structure of the agent library is illustrated in Fig. 3.

It is worth noting that the proposed library structure aims to provide a structured, platform-agnostic schema for organizing and sharing LLM agents for building energy analysis, focusing on usability and practical adoption by the community rather than serving as a centrally managed repository.

3. Case study

Four LLM agents or agentic workflows are developed and presented as case studies. As shown in Table 3, the case studies demonstrate various applications of LLMs in building energy analysis and modeling, including automating data-driven and physics-based building energy modeling, data visualization and analysis of building sensor/simulation data, as well as retrieval-augmented generation (RAG) for technical document knowledge extraction. The agents are designed using the developed agent schema and implemented across three mainstream LLM agent development platforms: LangGraph, AutoGen, and CrewAI,

```
{
  "id": "get_building_weather_data",
  "description": "This workflow is designed to get weather data based on user's description on building location",
  "model": "gpt-4o",
  "nodes": [
    {
      "id": "agent_node_1",
      "description": "",
      "system_message": "Question: Provide your building location, whether in zip code or address, and ask for any weather-related data. Thought: based on user's input and the input from action node, call the weather-API tool, or end the agent. Action: call the weather-API-tool. Observation: whether the user's data request is done",
      "type": "agent",
      "agent_tools": [],
      "short_term_memory": ["all"],
      "human_input": ["first_time"]
    },
    {
      "id": "action_node_1",
      "description": "",
      "system_message": "decide whether the tool 'weather-API' needs to be called. If no, END",
      "type": "action",
      "agent_tools": ["weather-API"],
      "short_term_memory": ["all"],
      "human_input": ["never"]
    }
  ],
  "edges": [
    {
      "id": "edge_1",
      "type": "simple",
      "connection": ["START", ["agent_node_1"]]
    },
    {
      "id": "edge_2",
      "type": "conditional",
      "connection": ["agent_node_1", ["action_node_1", "END"]]
    },
    {
      "id": "edge_3",
      "type": "simple",
      "connection": ["action_node_1", ["agent_node_1"]]
    }
  ],
  "tools": [
    {
      "id": "weather-API",
      "description": "",
      "content": "from langchain_community.tools.tavily_search import TavilySearchResults"
    }
  ],
  "long_term_memory": ["none"]
}
```

Listing 1. Example JSON file to define the agentic workflow shown in Fig. 2.

```

from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
import operator
from langchain_core.messages import
AnyMessage, SystemMessage, HumanMessage,
ToolMessage

# Placeholder to import tools and create tool list.
Example:
from tools.tool_x import tool_x
from existing_agent_tool_library import tool_y
...
tool_list = [tool_x, tool_y, ...]

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage],
operator.add]

class Agent:

    def __init__(self, model, tools, system=""):
        self.system = system
        graph = StateGraph(AgentState)

        # Placeholder to create agent nodes and
        action nodes. Example:
        graph.add_node("agent_node_1",
self.call_agent_node_1_function)
        graph.add_node("action_node_1",
self.take_action_function)

        # Placeholder to create edges (simple edge
        and conditional edge). Example:
        graph.add_conditional_edges(
            "agent_node",
            self.exists_action,

            {True: "action_node_1", False: END}
        )
        graph.add_edge("action_node_1",
"agent_node_1")

        # Placeholder to set entry point. Example:
        graph.set_entry_point("agent_node_1")

        self.graph = graph.compile()
        self.tools = {t.name: t for t in tools}
        self.model = model.bind_tools(tools)

        def exists_action(self, state: AgentState):
            result = state['messages'][-1]
            return len(result.tool_calls) > 0

        # Placeholder to define the function to call
        agent. Example:
        def call_agent_node_1_function(self, state:
AgentState):
            messages = state['messages']
            if self.system:
                messages =
[SystemMessage(content=self.system)] +
messages
            message = self.model.invoke(messages)
            return {'messages': [message]}

        def take_action_function(self, state:
AgentState):
            tool_calls = state['messages'][-1].tool_calls
            results = []
            for t in tool_calls:
                print(f"Calling: {t}")
                if not t['name'] in self.tools: # check for
bad tool name from LLM
                    print("\n ...bad tool name....")
                    result = "bad tool name, retry" #
instruct LLM to retry if bad
                else:
                    result =
self.tools[t['name']].invoke(t['args'])
            results.append(ToolMessage(tool_call_id=t['id'],
name=t['name'], content=str(result)))
            print("Back to the model!")
            return {'messages': results}

        # Placeholder for system prompt. Example:
        prompt = """You are a smart building energy
analysis. You can generate and debug code to do
analysis and modeling based on users' requests"""

        # Placeholder to define LLM foundation model.
        Example:
        from langchain_openai import ChatOpenAI
        model = ChatOpenAI(model="gpt-3.5-turbo")

        abot = Agent(model, tool_list, system=prompt)

        # Placeholder to define LLM foundation model.
        Example:
        user_input = "Help me generate an energy usage
trend line plots for the building data stored locally:
data/building_data.csv"

        messages = [HumanMessage(content=
user_input)]
        result = abot.graph.invoke({'messages':
messages})

```

Listing 2. Code template under LangGraph structure.

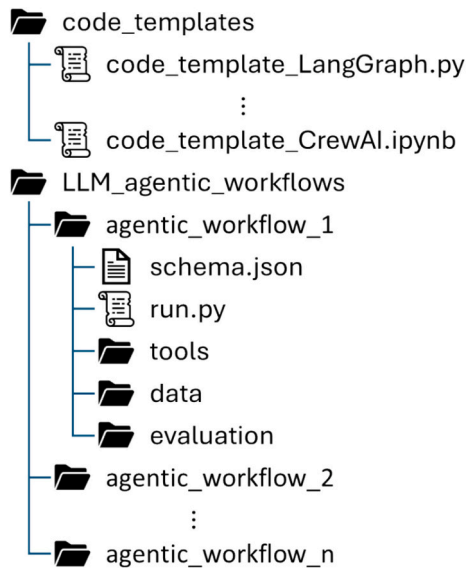


Fig. 3. Agent library structure shown as the GitHub repo snapshot.

showcasing the schema's adaptability across platforms. To illustrate the diversity of LLM agent structure types, each case study features a distinct structure: reflection, sequential, hierarchical, and RAG. In all case studies, tools are utilized through a structured mechanism where the planning agent triggers an action node linked to the appropriate tool, executing the required function. This ensures a consistent workflow across different tasks.

For each case study, the paper 1) describes the agent's function and

visualizes its structure with a diagram, 2) provides the corresponding JSON schema file, and 3) generates, analyzes, and discusses the results.

3.1. Case study 1: Data-driven building energy modeling

In Case Study 1, an LLM agent is introduced and developed to automatically create data-driven and machine-learning building energy models based on the user's specific needs. The agent output is a saved machine learning model that can be reused in the future. The user is required to 1) provide the data in CSV format, stored locally where the LLM agent can access it, and 2) specify detailed requirements for the modeling, such as: i) the model output, ii) the forecasting horizon, and iii) the preferred machine learning algorithms. This agent is important in terms of scaling the application of data-driven building energy modeling in various buildings with very few expert knowledge in machine learning and building science, which will eventually promote AI-based smart building operations and improve energy efficiency in the building sector.

Reasoning and Acting (ReAct) is a framework that combines reasoning (logical thinking) and acting (taking actions based on reasoning) to improve the decision-making process of AI models [31]. It enables the model to break down tasks, reason through solutions, and perform actions or generate responses accordingly. For instance, in a ReAct model, a chatbot might reason through a series of steps (e.g., checking a user's question for context) before performing an action like retrieving information from a knowledge base or making a decision.

Based on the agent's needs, the paper adopts a ReAct-based structure to ensure that the programming code is iteratively generated, debugged, and ultimately finalized correctly. Typically, the code does not function correctly on the first attempt, necessitating several steps: 1) executing the code, 2) identifying errors, 3) debugging, and 4) re-executing the code until successful model generation. This iterative process makes the

Table 3

Summary of case studies.

No.	Agent Description	Agent Development Platform	Agent Structure Type	Input	Output
Case Study 1	Data-driven building energy modeling automation	LangGraph	Reflection	Data for modeling and model requirement	Error-free Python code that leads to the generation of a machine learning model
Case Study 2	Physics-based building energy modeling automation	LangGraph	Sequential	Building description in text format	IDF file as EnergyPlus model input
Case Study 3	Data visualization and analysis of building sensor/simulation data	AutoGen	Supervisor/ Hierarchical	Description about the visualization	Error-free Python code that leads to the generation of a plot
Case Study 4	RAG of technical documents	CrewAI	Sequential structure with tools for RAG	Question about the document	Answer to the question

<pre> { "id": "data_driven_building_energy_modeling", "description": "This workflow is designed to build data-driven building energy models based on the data stored locally", "model": "gpt-4o", "nodes": [{ "id": "agent_node_1", "description": "", "system_message": "You run in a loop of Thought, Action, Observation.\n\nAt the end of the loop you output an answer.\n\nImagine you are a machine learning engineer who has to generate and execute Python code for machine learning modeling with the following 7 steps.\n\nStep 1 Data Preparation: Prepare data based on the provided two CSV file names with path for input and output data. Clean the data and remove missing values.\n\nStep 2 Feature Selection: Select important features based on correlation analysis, recursive feature elimination, principal component analysis, wrapper method, or filter method. This step is optional.\n\nStep 3 Data Splitting: Divide the dataset into three distinct subsets: the training set, validation set, and test set.\n\nStep 4 Model Selection: Choose suitable machine learning algorithm such as linear regression, decision tree, and random forest based on the validation results.\n\nStep 5 Hyper Parameter Tuning: Use optimizing model hyperparameters such as grid search or random search to finalize the model.\n\nStep 6 Model Evaluation: Test the model based on testing data based on any relative error metrics. Visualize the model performance using scatter plots and save it locally.\n\nStep 7 Save the model locally.\n\n\nUse the following format:\n\nQuestion or Request: create a machine learning model based on local csv files as inputs and output of the machine learning model.\n\nThought: you should always think about what to do, but basically you will follow the 7 </pre>	<pre> steps above\n\nAction Input: the generated Python code to be executed\n\nAction: execute the Python code based on \"python_repl\", the Python code execution tool\n\nObservation: the result of the action, whether the code is successfully executed without error or not\n\n... (this Thought/Action/Action Input/Observation can repeat N times)\n\nThought: The code is successfully executed without error and the machine learning model is successfully saved locally\n\nFinal Answer: the final models are saved at which location\n\n\nExample sessions:\n\nQuestion: I want to build a machine learning model. The input of the model is from a local csv file:\n\n/content/drive/MyDrive/LLM_data_driven_modeling/input_fx.csv, and the output is also from a local csv file:\n\n/content/drive/MyDrive/LLM_data_driven_modeling/output_fx.csv\n\nThought: I should follow the 7 steps including Data Preparation, Feature Selection, Data Splitting, Model Selection, Hyper Parameter Tuning, Model Evaluation, and Save the model locally.\n\nAction: generate Python code and use \"python_repl\" tool to execute the code\n\nObservation: There is an error in the code execution. Should debug and improve the code\n\nAction: improve Python code and use \"python_repl\" tool to execute the code\n\nObservation: There is no error in the code execution, and the machine learning model is successfully saved locally\n\nAnswer: The machine learning model is successfully developed and saved locally.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["first_time"] }, { "id": "action_node_1", "description": "", </pre>	<pre> "system_message": "Decide if the tool needs to be called. If not, END", "type": "action", "agent_tools": ["PythonREPL"], "short_term_memory": ["all"], "human_input": ["never"] }], "edges": [{ "id": "edge_1", "type": "simple", "connection": ["START", "agent_node_1"] }, { "id": "edge_2", "type": "conditional", "connection": ["agent_node_1", ["action_node_1", "END"]] }, { "id": "edge_3", "type": "simple", "connection": ["action_node_1", ["agent_node_1"]] }], "tools": [{ "id": "PythonREPL", "description": "this tool can take in Python code as string and execute the code, which is a built-in function called PythonREPL from LangChain", "content": "tools/python_repl.py" }], "long_term_memory": ["none"] } </pre>
---	--	--

Listing 3. JSON file based on the developed agent schema to define the LLM agent for Case Study 1.

ReAct structure particularly suitable. The workflow requires users to specify the data location. With this data, the agent node initially generates a thought and creates a preliminary draft of Python code. It then approaches conditional edges to decide whether to halt or proceed to the action node. Since this is the initial run and no model has yet been generated, the conditional edge directs to the action node. This node activates a Python code executor, specifically the PythonREPL from LangChain's tool package, to run/execute the code. Regardless of whether the code execution is successful, the results are returned to the agent node to begin another iteration. If the code runs without errors,

the agent node will reach the conditional edge and decide to stop as the task is completed. The JSON-based agent schema for this case study is shown in Listing 3. The diagram of the developed agent structure for Case Study 1 is shown in Fig. 4. The developed agent can be accessed through the agent library (https://github.com/lz356/building_llm_agent_library/tree/main/LLM_agentic_workflows/data_driven_building_energy_modeling). In the case study, the tested user input is "Generate a machine learning model to predict next hour's output (energy)". The input data of the model is '/content/drive/MyDrive/data-driven_building_energy_modeling/input.csv' and the output data is

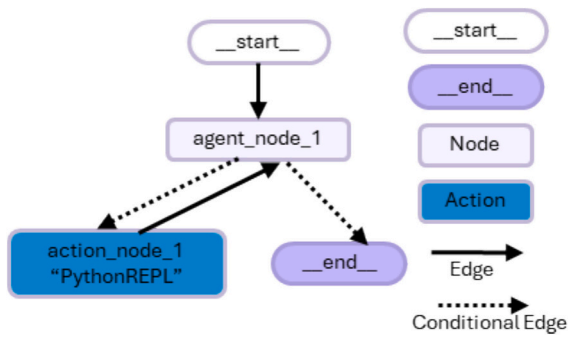


Fig. 4. Diagram of the developed agent structure for Case Study 1.

‘/content/drive/MyDrive/data_driven_building_energy_modeling/output.csv’.” The CSV files are auto-downloaded from the run.ipynb(https://github.com/lz356/building_llm_agent_library/blob/main/LLM_agentic_workflows/data_driven_building_energy_modeling/run.ipynb). The results of the agent with final Python code and performance visualization are shown in Listing 4, demonstrating that the developed agent follows the agent node's planning to generate and execute Python code, subsequently creating and saving the model along with its performance visualization. Before generating the error-free Python code, the agent encountered an issue where it failed to correctly read the input and output columns from the original dataset. This required one round of debugging to resolve. Due to space limitations, we do not present the full debugging process in detail. The generate machine learning model accuracy can be further improved by modifying the action node to improve the Python code quality until certain model accuracy is met.

3.2. Case study 2: Physics-based building energy modeling

In Case Study 2, an LLM agent is developed to automate several procedures in EnergyPlus [2], the flagship tool for physics-based building energy modeling. The LLM agent processes the building description in natural language as input, and outputs a list of IDF objects, which are the standard text-based EnergyPlus input format. The application of the agent is important to make EnergyPlus tool more accessible to people who do not know complex EnergyPlus software but still can do

quantitative energy analysis to design and renovate buildings. With this agent, EnergyPlus modelers can also save significant time by avoiding manual data entry from building description and specification into EnergyPlus.

A sequential or flow-engineering structure, which includes a series of connected steps, is utilized to achieve the goal. The agent structure, depicted in Fig. 5, follows a linear, step-by-step workflow. The users need to provide a building description or specification, covering various aspects such as construction, internal load, and HVAC systems, along with any specific requirements for these elements. The first agent node processes this input, conducting unit conversions to standardize the units across the description, which is critical for compatibility with EnergyPlus. The second agent node decides what IDF objects to create and extracts unstructured texts about these IDF objects from the description. Finally, the third agent node generates structured text-based IDF objects for each object to be created, adhering to the syntax required for each object. This sequential or flow-engineering workflow is straightforward, with no use of tools or conditional edges in this

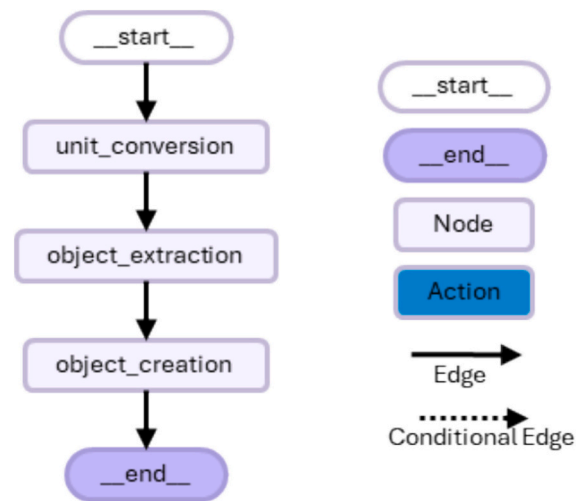
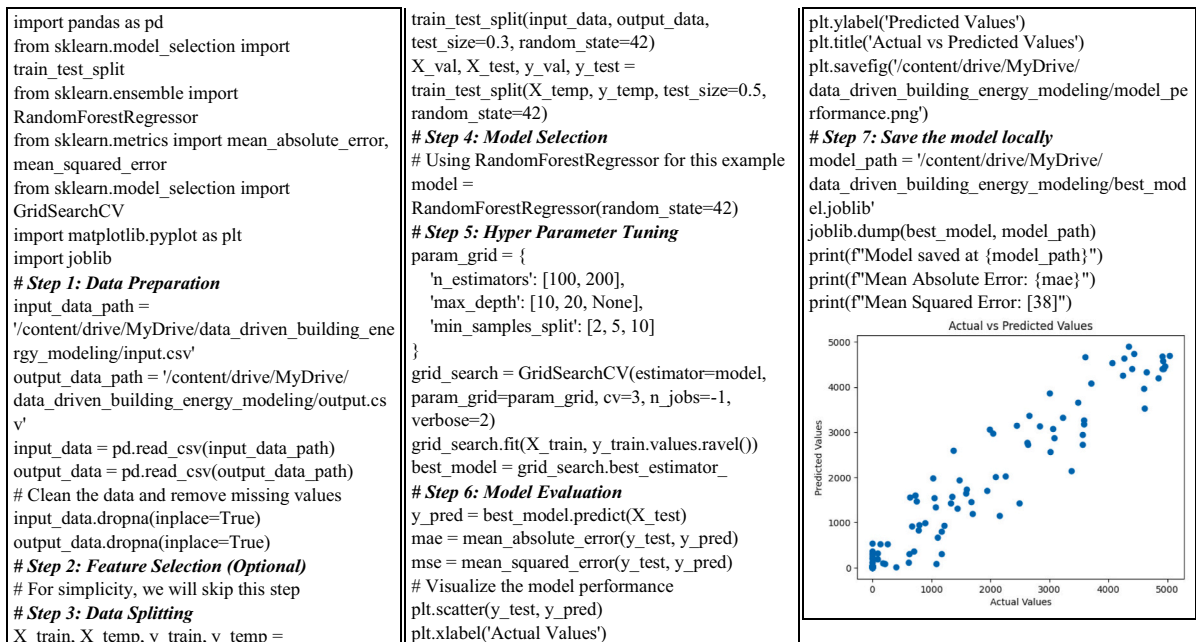


Fig. 5. Diagram of the developed agent structure for Case Study 2.



Listing 4. Generated error-free Python code and executed model performance visualization for Case Study 1.

<pre>{ "id": "physics_based_building_energy_modeling", "description": "This workflow is designed to assist physics-based building energy models based on building description", "model": "gpt-4o", "nodes": [{ "id": "unit_conversion", "description": "unit conversion", "system_message": "This agent node updates the building description to use the correct SI units (International System of Units) while keeping the rest of the description unchanged.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["first_time"] }, { "id": "object_extraction", "description": "object extraction", "system_message": "This agent node processes building description under SI unit to create an object dictionary about 1) material/construction, 2) internal load (people, lights, electric equipment, infiltration), and 3)</pre>	HVAC system. The dictionary's keys are the EnergyPlus IDF object type and its name to be created in tuple format (object type in str, object name in str, object name cannot be the same as object type), and the values to be detailed description or requirement of the IDF object, in string format.", <pre> "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "object_creation", "description": "object creation", "system_message": "This agent node generates all text-based idf objects for EnergyPlus simulations from the extracted object dictionary. Some general rules when generating the object: Start with filling out default values for all fields. Fill out or replace the value that can be extracted from the object dictionary. - If the field is not mandatory and is not defined by the specific requirement, leave the value blank in the field. - If the field is mandatory but not defined by the specific requirement, set it to 'TBD' (to be defined). - Even though some fields are optional or not defined by the specific requirement, keep</pre>	them in the object and do not remove them. - In the generated object, comment after exclamation mark for each field about why the value is set or is left blank.", <pre> "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] },], "edges": [{ "id": "edge_1", "connection": ["agent_node_1", ["agent_node_2"]] }, { "id": "edge_2", "connection": ["agent_node_2", ["agent_node_3"]] },], "tools": [], "long_term_memory": ["none"] }</pre>
--	--	---

Listing 5. JSON file based on the developed agent schema to define the LLM agent for Case Study 2.

<pre>! 1. Constructions Material, RodentBarrier_HDPE, !- Name Smooth, !- Roughness 0.00127, !- Thickness 0.500, !- Conductivity 950, !- Density 1820; !- Specific Heat Material, InsulatedJoistLayer !- Name Smooth, !- Roughness 0.36733, !- Thickness 0.048, !- Conductivity 101.4, !- Density 986; !- Specific Heat Material, OSB, !- Name Smooth, !- Roughness 0.04445, !- Thickness 0.119, !- Conductivity 544, !- Density 1210; !- Specific Heat Material, Plywood, !- Name Smooth, !- Roughness 0.00635, !- Thickness 0.115, !- Conductivity 544, !- Density</pre>	<pre>1210; !- Specific Heat Construction, FloorConstruction, !- Name RodentBarrier_HDPE, !- Layer 1 InsulatedJoistLayer, !- Layer 2 OSB, !- Layer 3 Plywood; !- Layer 4 ! 2. Internal Loads People, iUnitPeople, !- Name iUnitZone, !- Zone or ZoneList Name People, !- Calculation Method 2, !- Number of People , !- People per Zone Floor Area , !- Zone Floor Area per Person 0.3, !- Fraction Radiant ... ! 3. Infiltration ZoneInfiltration:DesignFlowRate, iUnitInfiltration, !- Name iUnitZone, !- Zone or ZoneList Name Always On, !- Schedule Name Flow/Area, !- Design Flow Rate Calculation Method , !- Design Flow Rate</pre>	<pre>0.03, !- Flow per Zone Floor Area , !- Flow per Exterior Surface Area , !- Air Changes per Hour 1, !- Constant Term Coefficient 0, !- Temperature Term Coefficient 0, !- Velocity Term Coefficient 0; !- Velocity Squared Term Coefficient ! 4. HVAC System HVACTemplate:Zone:PTAC, iUnitPTAC, !- Name iUnitZone, !- Zone Name Always On, !- Thermostat Schedule autosize, !- Cooling Supply Air Flow Rate autosize, !- Heating Supply Air Flow Rate autosize, !- No Load Supply Air Flow Rate ... (due to the long people object and HVAC template object and the limited space in the paper, we just copy the first few important field names and values in this listing, and the complete results can be found in the agent library https://github.com/lz356/building_ilm_agent_library/tree/main/LLM_agentic_workflows/physics_based_building_energy_modeling)</pre>
---	---	---

Listing 6. IDF objects extracted from building description as the final output of the LLM agent in Case Study 2.

particular study. The schema of this agent is shown in Listing 5. Other details of the agent can be found in the agent library: https://github.com/lz356/building_ilm_agent_library/tree/main/LLM_agentic_workflows/physics_based_building_energy_modeling.

In the case study, the input of the case study is a building description "Building Name: iUnit. 1. Constructions. 1.1. Floor. Floor Layer 1 (Outside): Rodent Barrier (HDPE); Roughness: Smooth; Thickness: 0.05 in; Conductivity: 3.472 Btu-in/hr-ft²-R; Density: 59.3 lb/ft³; Specific Heat: 0.4347 Btu/lb-R; Floor Layer 2: Insulated Joist Layer; Roughness:

Smooth; Thickness: 14.462 in; Conductivity: 0.3323 Btu-in/hr-ft²-R; Density: 6.336 lb/ft³; Specific Heat: 0.2357 Btu/lb-R. 2. Internal Loads. People. Internal load from people follows the NREL's business hours, 1) Monday–Thursday, 8 a.m.–5 p.m., 2) Friday, 8 a.m.–4 p.m., 3) Closed on Saturday and Sunday. There are on average two persons in iUnit during business hours and no people in time of the rest. 3 Infiltration. The infiltration will be defined using IDF object "ZoneInfiltration:DesignFlowRate". In this object, according to the onsite measurements, I = 1, Fschedule = always on (1,1,1,1,...), A = 0.03, B = 0.003, C = 0, D = 0.4.

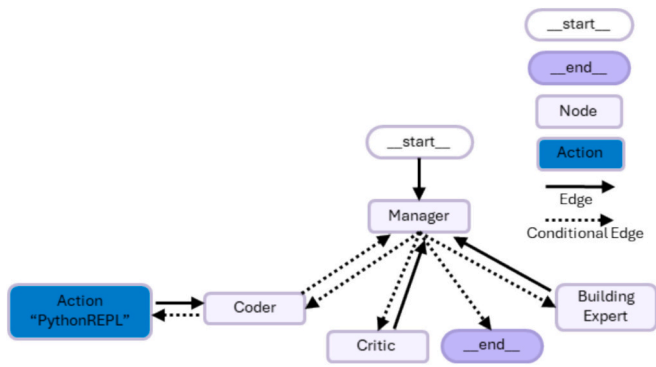


Fig. 6. Diagram of the developed agent structure for Case Study 3.

HVAC System. Use HVAC template to design and automate packaged terminal air conditioner (PTAC) systems with hot water boiler". The results shown in Listing 6 indicates a generally successful unit conversion, object extraction, and object generation. However, some errors are identified: 1) schedule object is not created at all, 2) there are reference issues due to the lack of information, and 3) some mandatory fields are left blank.

The evaluation of the IDF objects generated, including metrics such as EnergyPlus execution success rates and simulation accuracy, is detailed in a separate publication [38].

3.3. Case study 3: Data visualization of building sensor/simulation data

In Case Study 3, an LLM agent is developed to automatically perform data visualization and analysis of building sensor or simulation data

<pre> { "id": "data_visualization_of_building_sensor_simulation_data", "description": "This workflow is designed to automatically perform data visualization and analysis of building sensor or simulation data based on the user's specific needs.", "model": "gpt-4o", "nodes": [{ "id": "Manager", "description": "Distributes tasks among other agents and manages the overall workflow.", "system_message": "You are a manager responsible for overseeing data visualization and analysis of building sensor or simulation data based on user-specific requirements. Distribute tasks to the Coder, Critic, and Building Expert as needed. If the Critic suggests 'TERMINATE,' conclude the workflow.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["first_time"] }, { "id": "Coder", "description": "Generates and executes code for data visualization and analysis.", "system_message": "You are responsible for generating Python code based on the Manager's requests. Decide whether to call tools to execute Python code or consult with the Manager for clarification.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "action_node_1", "description": "Executes Python code generated by the Coder and returns the results.", "system_message": "Call the PythonREPL tool to execute the Python code generated by the Coder, and return the results.", "type": "action", "agent_tools": ["PythonREPL"], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "Critic", "description": "Evaluates the quality of visualization code using best practices.", "system_message": "Critic. You are a helpful assistant highly skilled in evaluating the quality of a given visualization code by providing a score from 1 (bad) - 10 (good) while providing clear rationale. YOU MUST CONSIDER VISUALIZATION BEST PRACTICES for each evaluation. Specifically, you can carefully evaluate the code across the following dimensions:\n\nbugs (bugs): Are there bugs, logic errors, syntax errors, or typos? Are there any reasons why the code may fail to compile? How should it be fixed? If ANY bug exists, the bug score MUST be less than 5.\n- Data transformation (transformation): Is the data transformed appropriately for the visualization type? E.g., is the dataset appropriately filtered, aggregated, or grouped if needed? If a date field is used, is the date field first converted to a date object, etc.?\n- Goal compliance (compliance): How well does the code meet the specified visualization goals?\n- Visualization type (type): CONSIDERING BEST PRACTICES, is the visualization type appropriate for the data and intent? Is there a visualization type that would be more effective in conveying insights? If a different visualization type is more appropriate, the score MUST BE LESS THAN 5.\n- Data encoding (encoding): Is the data encoded appropriately for the visualization type?\n- Aesthetics (aesthetics): Are the aesthetics of the visualization appropriate for the visualization type and the data?\n\nYOU MUST PROVIDE A SCORE for each of the above dimensions.\n{bugs: 0, transformation: 0, compliance: 0, type: 0, encoding: 0, aesthetics: 0}\nDo not suggest code.\nFinally, based on the critique above, suggest a concrete list of actions that the coder should take to improve the code. Return 'TERMINATE' if the figure meets the critique and is successfully generated.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "Building_Expert", "description": "Provides domain knowledge to enhance visualization and analysis.", "system_message": "You are a building expert with extensive domain knowledge in building energy analysis. Provide expert insights to enhance the visualization and analysis of building sensor or simulation data.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }], "edges": [{ "id": "edge_1", "type": "simple", "connection": ["START", ["Manager"]] }, { "id": "edge_2", "type": "conditional", "connection": ["Coder", ["Manager", "action_node_1"]] }, { "id": "edge_3", "type": "simple", "connection": ["action_node_1", ["Coder"]] }, { "id": "edge_4", "type": "simple", "connection": ["Critic", ["Manager"]] }, { "id": "edge_5", "type": "simple", "connection": ["Building_Expert", ["Manager"]] }, { "id": "edge_6", "type": "conditional", "connection": ["Manager", ["Coder", "Critic", "Building_Expert", "END"]] }], "tools": [{ "id": "PythonREPL", "description": "Executes Python code as a string using the PythonREPL tool from LangChain.", "content": "tools/python_repl.py" }], "long_term_memory": ["none"] } </pre>	<pre> { "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "edge_1", "type": "simple", "connection": ["START", ["Manager"]] }, { "id": "edge_2", "type": "conditional", "connection": ["Coder", ["Manager", "action_node_1"]] }, { "id": "edge_3", "type": "simple", "connection": ["action_node_1", ["Coder"]] }, { "id": "edge_4", "type": "simple", "connection": ["Critic", ["Manager"]] }, { "id": "edge_5", "type": "simple", "connection": ["Building_Expert", ["Manager"]] }, { "id": "edge_6", "type": "conditional", "connection": ["Manager", ["Coder", "Critic", "Building_Expert", "END"]] }, { "id": "PythonREPL", "description": "Executes Python code as a string using the PythonREPL tool from LangChain.", "content": "tools/python_repl.py" } </pre>
---	---

Listing 7. JSON file based on the developed agent schema to define the LLM agent for Case Study 3.

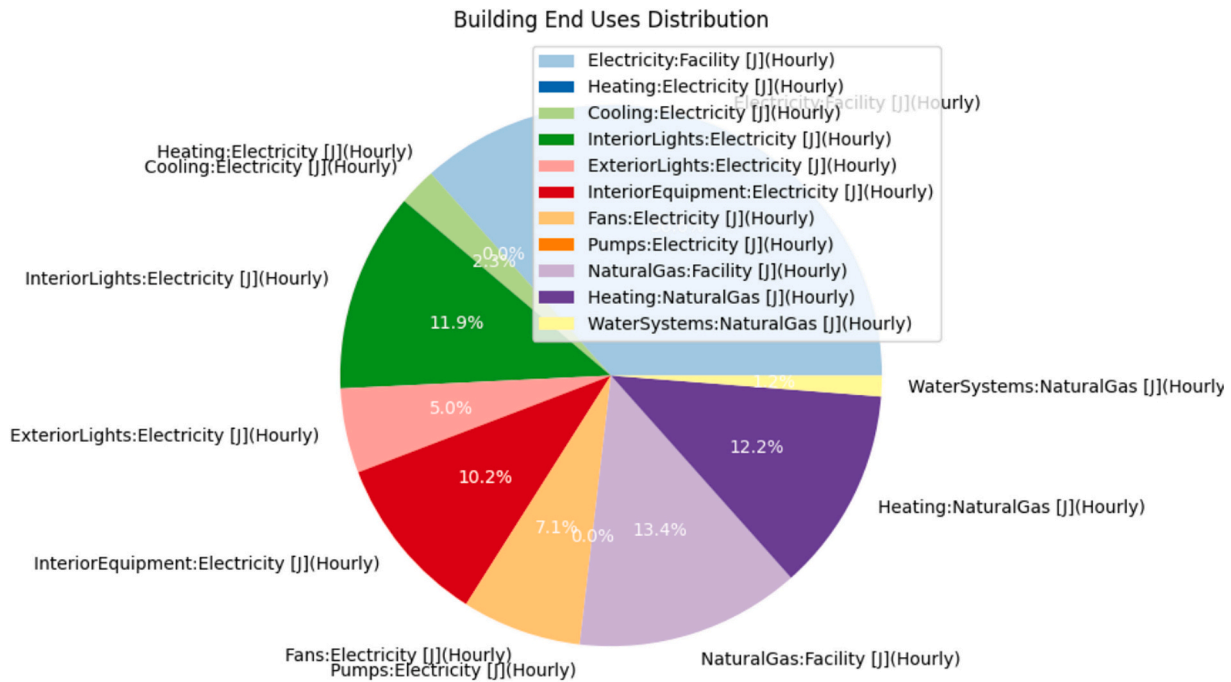


Fig. 7. Generated visualization for an EnergyPlus simulation output.

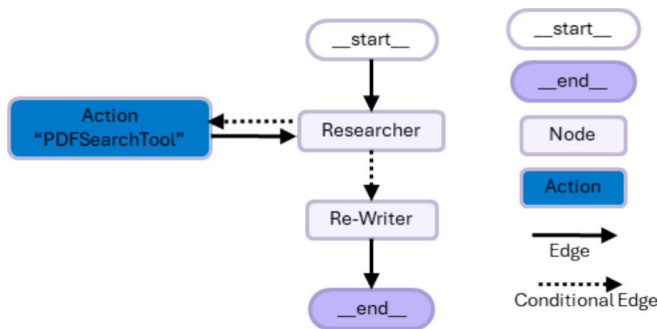


Fig. 8. Diagram of the developed agent structure for Case Study 4.

based on the user's specific needs. The agent's input consists of structured building data, either from sensors or simulations, along with the user's request specifying whether they require visualization, analysis such as sensitivity analysis, trend prediction, and anomaly detection.

The expected output of the agent includes Python code, executed results with corresponding figures and numbers, and insights addressing the user's request. This agent is significant as it greatly reduces the time, effort, and expertise required to quickly and automatically understand the operation, status, or faults of a building. It benefits building operators, researchers, and anyone relying on data as a critical input for their work.

The agent structure graph is depicted in Fig. 6. A supervisor structure, which includes a "Manger" agent overseeing and coordinating the activities of other agents, including "Coder", "Critic", "Building Expert" agents, is utilized to achieve the goal. The schema of this agent is shown in Listing 7. The users need to provide where the data is located and where the visualization figure is located, as well as the key information about what

visualization the user is expecting. They can be explicit about what figure type and data to use, or they can just be very vague and let the agentic workflow to decide the details. This hierarchical structure is defined under our developed agent schema and the detailed JSON file is listed in Listing 7. The Manger agent use a conditional edge to link to all the other agents shown in dotted lines to arrange the task, and when the task is assigned to a certain agent, it will return the results back to the Manager agent shown in solid lines. Also, the coder agent is connected with the PythonREPL tool to execute and code as the Manger agent demands. Other details of the agent can be found in the agent library: https://github.com/lz356/building_llm_agent_library/tree/main/LLM_agentic_workflows/data_visualization_of_building_sensor_simulation_data.

In this case study, as an example, the author used EnergyPlus simulation output to visualize pie chart of energy enduses such as heating, cooling, fan, lighting, etc. The input for the agentic workflow is: "get data locally from '/content/drive/MyDrive/LLM_Data_Analysis_and_Visualization/RefBldgSmallOfficePost1980_v1.4.7.2_2B_U-SA_AZ_PHOENIX.csv' and plot a visualization of pie chart for enduses of the building. Save the generated figure to '/content/drive/MyDrive/LLM_Data_Analysis_and_Visualization/'." And Fig. 7 shows the final generated pie chart of the visualization. It can be seen that the pie chart correctly identifies all the enduse among many other columns such as weather and indoor environments. Then it successfully creates the visualization with correct legend. An error is identified that it does not recognize "Electricity:Facility [J] (Hourly)" as the total electricity and mistakenly include this as enduses.

The results of the agent with final Python code and visualization, demonstrating that the developed agent follows the agent node's planning to generate and execute Python code, subsequently creating and saving the figure. The agent is also equipped to debug errors effectively. The generated figure accuracy can be further improved by injecting more knowledge about column convention of EnergyPlus output, so that it can successfully identify more accurate enduse columns.

<pre> { "id": "retrieval_augmented_generation_of_technical_documents", "description": "This workflow employs a retrieval-augmented generation (RAG) approach to automatically visualize and analyze building sensor or simulation data based on the user's specific requirements. It reduces the time and expertise needed to interpret complex technical documents (e.g., the EnergyPlus Engineering Reference) and provides professional, domain-informed insights.", "model": "gpt-4o", "nodes": [{ "id": "Researcher", "description": "Searches the technical document to find accurate and relevant information in response to the user's query.", "system_message": "You are the Researcher. Upon receiving a user's question, search through the provided technical document to find relevant passages and factual details. Return clear, concise findings without additional commentary beyond what is found in the source.", "type": "action", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["first_time"] }, { "id": "Re-Writer", "description": "Refines the Researcher's </pre>	<pre> findings into a polished, accessible answer understandable by non-experts.", "system_message": "You are the Re-Writer. Take the Researcher's extracted information and rewrite it in a professional, clear, and accessible manner. Your goal is to ensure the final answer is both accurate and easily understood by non-building experts. Explain technical terms as needed and maintain a coherent, easy-to-follow narrative.", "type": "agent", "agent_tools": [], "short_term_memory": ["all"], "human_input": ["never"] }, { "id": "action_node_1", "description": "", "system_message": "Decide to call the tool PDFSearchTool to retrieve relevant information from PDF, or decide to pass the existing results to Re-Writer agent", "type": "action", "agent_tools": ["PDFSearchTool"], "short_term_memory": ["all"], "human_input": ["never"] }], "edges": [{ "id": "edge_1", "type": "simple", "connection": ["START", ["Researcher"]] }, { "id": "edge_2", "type": "conditional", "connection": ["Researcher", ["action_node_1", "Re-Writer"]] }, { "id": "edge_3", "type": "simple", "connection": ["action_node_1", ["Researcher"]] }, { "id": "edge_4", "type": "simple", "connection": ["Re-Writer", ["END"]] }], "tools": [{ "id": "PDFSearchTool", "description": "Extracts and searches text within PDFs, supports keyword queries, phrase matching, metadata filtering, and OCR for scanned documents. Ideal for analyzing and retrieving insights from PDFs efficiently.", "content": "from crewai_tools import PDFSearchTool" }], "long_term_memory": ["none"] } </pre>
---	---

Listing 8. JSON file based on the developed agent schema to define the LLM agent for Case Study 4.

3.4. Case study 4: Retrieval-augmented generation (RAG) of technical documents

RAG is a technique that enhances the performance of LLMs by dynamically incorporating external knowledge from sources such as document repositories, databases, or search engines during inference, thereby improving factual accuracy and grounding responses in relevant context [39]. It retrieves relevant information from these sources and uses it to generate more accurate and informative responses. Moreover, A RAG-based model could answer a question about the latest scientific research by first retrieving relevant journal articles and then generating a response based on the content it retrieved. In Case Study 4, a RAG approach is implemented using a multi-agent system to efficiently retrieve and refine information from a PDF reference document based on a user's specific query. The workflow, illustrated in Fig. 8., begins with user input comprising a query and a designated PDF. The system generates output that includes precisely sourced content, re-written in clear and accessible language.

At the core of this approach is the Researcher agent, which connects with the Action agent to utilize the PDFSearchTool for extracting relevant information. The extracted content is then refined by the Re-Writer agent, ensuring enhanced clarity and user-friendliness. The workflow streamlines the process of locating, interpreting, and presenting complex technical information, significantly reducing the time and expertise required. This solution benefits researchers, practitioners, and others who depend on technical data for their work. Further details about this agentic workflow are outlined in the developed schema shown in Listing 8.

In this case study, the author uses the EnergyPlus Engineering Reference, a widely recognized and authoritative technical document that provides detailed explanations of the calculations, modeling methodologies, and system components employed in the EnergyPlus building energy simulation software. The author tested the question of

“What is Baffle Heat Balance?”, and Listing 9 shows the step-by-step process of agentic workflow. The results show that the “Researcher” agent has multiple interaction with the “PDFSearchTool” and finalize the content extract from PDF. Then the “Re-Writer” agent just converts the writing to a more understandable form, for example: “To simplify, this equation allows us to understand how the baffle interacts thermally with its environment, ensuring that all incoming and outgoing thermal energy is accounted for and balanced, which ultimately determines its surface temperature.” The successful case study shows that the agentic workflow successfully conducts RAG and postprocessing to the RAG work to improve it with better explanation.

The results of the agent with final questions answered are shown in Listing 9 in Appendix, demonstrating that the developed agent follows the agent node's planning to extract information and generate correct answer to professional questions. The agent is also equipped to debug errors from tool call effectively by repeatedly call tools until good results are extracted from PDF.

4. Results and discussion

The four case studies demonstrate distinct applications and structures of LLM agents. Listing 4, Listing 6, Fig. 7, and Listing 9 respectively

Table 4

Evaluation Results Across Three Modeling Tasks with Different Datasets.

No.	Datasets	Completion Rate (10 runs)	Model Testing Error (CV-RMSE)
1	Reference commercial building	70 %	8.3 %
2	NREL campus building	50 %	14.5 %
Average		60 %	11.4 %

show the success run of four agents. Case Study 1 illustrates an agent functioning similarly to a data scientist, processing data and developing models efficiently. The output of Case Study 1 is error-free and executable Python code which leads to the successful modeling and storage of the model. In Case Study 2, the agent acts like a building energy modeler, adeptly handling unstructured building descriptions and successfully converting them into IDF objects while adhering to the pre-defined workflow. In Case Study 3, the agent functions as a data analyst, automatically generating error-free and executable Python code to perform data visualization and analysis of building sensor or simulation data. The output includes accurate visualizations, such as trend predictions and anomaly detections, effectively addressing the user's specific needs. In Case Study 4, the agent operates as a technical document interpreter, utilizing a RAG approach to extract precise information from reference documents and refine it into clear, accessible explanations tailored to user queries.

However, all case studies present single instances of agent run, raising questions about their future reliability as depicted in the study. In all applications of LLMs, maintaining self-consistency is a significant challenge. Self-consistency refers to the ability of the model to generate consistent and reliable results across different runs. To enhance this, one approach applied in the case study is to set the temperature parameter to zero or a small value, although uncertainties remain due to the inherent randomness of LLM outputs. Consequently, the library structure incorporates an evaluation folder for each agent. This component is designed to facilitate repeated testing of the agents across various scenarios, enhancing the assessment of their adaptability and stability. The evaluation process can also utilize feedback from both users and developers to determine an agent's effectiveness. By quantifying success rates and analyzing input-output records, this evaluation helps to clarify the types of tasks suitable for each agent and guides users in effectively leveraging their capabilities. Since this is an initial study, we intentionally adopt a flexible evaluation approach, which does not include a standardized evaluation framework due to the diversity of potential evaluation methods available. Instead, the evaluation is handled separately, allowing different assessment techniques to be applied based on specific requirements and contexts.

The paper conducts a comprehensive evaluation for Case Study 1 as shown in Table 4, as well as in the agent library (https://github.com/lz356/building_llm_agent_library/blob/main/LLM_agentic_workflows/data_driven_building_energy_modeling/evaluation/evaluation.ipynb). Table 4 summarizes the evaluation results of the agent, tested under two data-driven building energy models tasks, respectively based on two datasets. The datasets are 1) Dataset 1: the building data simulated from a reference small office building [40], and 2) Dataset 2: the building data collected from a real cafeteria building located at National Renewable Energy Laboratory. These datasets reveal different data sources, data quality, and building types. Each dataset was tested repeatedly 10 times, making a total of 20 runs.

The performance metrics are 1) completion rate which quantifies the rate that the agent eventually and successfully generate the data-driven building energy models, and 2) average model test error of each successfully generated model using the error metrics of Coefficient of Variation of the Root Mean Square Error (CV-RMSE). The results from Table 4 show that the completion rate and the accuracy of the modeling task based on the Dataset 1 simulation data is 70 % and 8.3 % respectively, which is better than that for Dataset 2 real building data (50 % and 14.5 %). The results are expected, as real building data often suffers from poorer data quality, complicating the data-driven modeling process and reducing accuracy. Overall, the agent can achieve an average of 60 % code completion rate and an average of 11.4 % test error across two modeling tasks, which aligns acceptably with ASHRAE's 15 % CV-

RMSE threshold for energy prediction. Notably, while LLM outputs inherently vary, increasing the number of test runs significantly enhances the success rate. For example, if the success rate for a single run is 60 % (average success rate of the case study), the probability of achieving at least one successful outcome across two independent runs rises to 84 %. Although this study conducted only ten runs per task, in practice, it is common to execute multiple runs and select the best-performing output. This approach ensures that even with inherent variability, the robustness of the developed agent is maintained. These findings validate the reliability of the agent and provide a valuable reference for both agent users and developers aiming to optimize performance in real-world applications. Due to space limitations and the specific focus of this paper, the author did not include a detailed comparison with other manual IDF generation methods. However, previous research has demonstrated the advantages of the agentic approach over manual IDF generation and simple prompt engineering methods, highlighting its superior efficiency and accuracy in handling complex workflows [28].

For Case Study 2, the general steps are executed correctly as designed by the agents, the generated IDF objects have accuracy issues, including wrong reference of objects and incorrect field value based on wrong assumption or oversimplifications. For instance, as shown in Listing 6, the Zone Name of iUnitZone in the "HVACTemplate:Zone" object is undefined, though it should be specified using a Zone object. Similarly, many schedule objects, such as Always On are missing. Additionally, essential objects like 'Version' and 'Building' are absent, rendering the generated IDF objects suitable only as references or drafts, rather than finalized versions ready for direct execution in EnergyPlus. This can be improved by 1) better prompt engineering in each agent node with domain knowledge to correctly instruct the LLMs, 2) using a domain-specific LLM foundation model, and 3) add an action node in the agent to iteratively validate and debug generated IDF object by calling an EnergyPlus execution tool that can actually run the generated IDF objects. Consequently, a similar evaluation process as in Case Study 1 has not been conducted for Case Study 2, due to challenges in quantifying accuracy. Addressing these improvements could be a focus for future research after these potential enhancements are implemented. Due to space limitations, detailed explanations of Case Studies 3 and 4 are not provided, as the focus is on the schema's ability to support complex agentic workflows. The results of Case Studies 3 and 4 demonstrate that the schema effectively handles hierarchical multi-agent collaboration and RAG scenarios.

Overall, the four case studies underscore the advantages of a structured and well-defined description of LLM agents according to the developed schema, which simplifies and may even automate agent development. The library empowers users to readily employ, share, customize, and evaluate agents, significantly diminishing the engineering effort required to use LLMs effectively and reliably. Despite these achievements, there are notable limitations to this work. Firstly, the case studies are limited to ReAct, flow engineering, hierarchical, and RAG structures, presenting an opportunity to expand to other workflows, such as planner-executor structures and memory-augmented structures. Secondly, the agent library currently features only four examples; a broader range of contributions from LLM engineers, building engineers, researchers, and the developer community is essential to enhance the diversity of the library. Thirdly, the memory of the agent is simplified in case studies, limiting its ability to retain and utilize context effectively across multiple interactions or sessions. Complex memory definitions are critical in scenarios where agents must process iterative user requests, track long-term dependencies, or dynamically adapt workflows based on prior outputs. For example, in Case Study 3, a more robust memory system could help store and retrieve previously analyzed

data trends or anomalies, allowing the agent to contextualize new queries and avoid redundant processing. Similarly, in Case Study 4, enhanced memory could enable the agent to retain intermediate results from previous queries on a technical document, reducing retrieval costs and improving response accuracy by narrowing the search scope. By ensuring that each agent accesses only the relevant amount of data necessary for the task, complex memory structures can improve the precision of outputs while simultaneously minimizing computational costs and resource usage. This highlights the importance of advancing memory design in future iterations of the framework.

The author developed agents in three agent platforms: LangGraph, AutoGen, and CrewAI. Compared to LangGraph, **AutoGen** offers a more streamlined approach to handling multi-agent collaboration with minimal code. Without the need to define nodes and edges, AutoGen simplifies complex workflows significantly. For instance, in Case Study 3, the “GroupChat” function enables automatic coordination of complex information flows among agents, eliminating the need for explicit communication configurations. This makes AutoGen especially advantageous for scenarios requiring efficient, low-effort implementation of agent collaboration and communication protocols. Additionally, AutoGen excels in its adaptability to dynamic workflows, offering users the ability to quickly deploy and adjust agent interactions without intricate setup or manual intervention [32]. **LangGraph**, on the other hand, provides a highly customizable framework for building agentic workflows, leveraging explicit node and edge definitions to map out intricate relationships and dependencies. This level of granularity is particularly beneficial in Case Study 2, where defining structured workflows and tracking dependencies between tasks is critical. LangGraph's explicit graph-based representation offers unmatched transparency and control, making it an excellent choice for projects requiring detailed workflow management and extensive customization. However, this strength also introduces additional setup complexity, as users must define and manage the graph manually, which can be time-consuming for simpler applications or users with less technical expertise [33]. In contrast, **CrewAI** is heavily focused on simplicity, but it sacrifices customization and adaptability. For example, in Case Study 4, the researcher agent can autonomously perform tool operations, such as invoking tools, retrieving information, and extracting results, without requiring explicit configurations from the user. While this approach reduces setup complexity for basic functionalities, it significantly limits the user's ability to modify workflows. CrewAI lacks the flexibility to accommodate more sophisticated use cases or highly specific agent behaviors, which makes it less suitable for scenarios requiring extensive customization or control. Moreover, it does not support multimodal inputs, provide a visual builder, or offer built-in deployment options, further restricting its applicability in complex or diverse workflows [34] [35]. **Overall**, in various applications of LLMs in building energy analysis and modeling with different agent structure types and inputs (as shown in Table 3), our developed agentic schema accommodates all three platforms—AutoGen, LangGraph, and CrewAI—by providing a unified framework for defining and describing agentic workflows. It enables seamless translation between platforms and the integration of workflows, such as combining one from AutoGen with another from LangGraph. Regardless of how agent platforms evolve, the schema serves as a flexible template for defining and sharing agentic workflows, no matter their complexity or the platform in which they are implemented.

5. Conclusions

This paper presented a structured schema and an open-source library designed to streamline the development, sharing, and deployment of LLM agents for building energy analysis and modeling. The proposed schema effectively organizes the complex workflows involved in LLM agentic processes, allowing for systematic reuse, customization, and evaluation across different projects and platforms. The four case studies

demonstrate the schema's versatility, with successful implementations in four very different applications. Although the library currently features a limited number of agents and the developed agents need improvements in accuracy, its foundation encourages collaboration and expansion by the research and development community. This work represents a significant step toward democratizing the systematic use and development of LLM technologies in the building energy sector, promoting efficiency, innovation, and broader adoption.

In terms of limitations, as the paper concentrates on structuring the agent schema and agent library, there is a considerable need to expand the types of agentic workflows, the number of case studies, the number of agents, and the environments for agent development within the library. Providing more example code for agent development will simplify the process for developers to find relevant cases that inspire and inform their own projects. Future enhancements should aim to refine the schema and the agent library structure to accommodate a broader array of agentic workflows.

Moving forward, the evolution of the agent library cannot be the sole responsibility of a single individual or research lab. It requires contributions from everyone involved in LLM use within building analysis and modeling, as well as from the AI community. Similar to how open-source platforms like Hugging Face have revolutionized access to LLM models, this agent library is designed to be open-sourced, encouraging the community to contribute and share their developments in a structured manner. However, managing such contributions, even within a structured framework, poses challenges, particularly in adapting to different developers who may have varying preferences and may not strictly adhere to established guidelines.

Future work will focus on expanding the validation of the proposed schema across a broader range of platforms beyond AutoGen, LangGraph, and CrewAI. This will involve implementing identical case studies across all selected platforms, demonstrating lossless schema transfer, and comparing performance across different implementations. Additionally, we plan to develop specific methods for cross-platform conversion to facilitate seamless adoption of the schema. A roadmap for achieving complete platform independence will be outlined, including potential platforms for further testing and validation. Besides AutoGen, LangChain, and CrewAI, there are other platforms/environments/tools for developing AI agents. For example, developed by Microsoft, TaskWeaver is a code-first framework for developing LLM-based autonomous agents [35]. And OpenAI Swarm is used to develop large-scale AI agents [34]. We plan to utilize them in our future case studies to validate our library across diverse environments and platforms. Another future initiative could include developing automated validation tools to ensure code compliance automatically with the structured schema. Additionally, exploring automated processes that can translate an agent schema into a fully operational agent without human intervention remains an intriguing prospect for further research. Finally, we plan to include more diverse building types to enhance the representativeness of the evaluation.

CRedit authorship contribution statement

Liang Zhang: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xiaoqin Fu:** Writing – original draft, Validation, Formal analysis. **Yanfei Li:** Writing – review & editing, Conceptualization. **Jianli Chen:** Writing – review & editing, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Appendix

# Agent: Research Agent ## Task: Answer the people's questions based on the content in PDF. The research agent will search through the PDF to find the relevant answers. Your final answer MUST be clear and accurate, based on the content of the PDF. Here is the people's question: What is Baffle Heat Balance? I encountered an error while trying to use the tool. This was the error: Arguments validation failed: 1 validation error for FixedPDFSearchToolSchema query Input should be a valid string [type=string_type, input_value={'description': 'Baffle H...Balance', 'type': 'str'}, input_type=dict] For further information visit https://errors.pydantic.dev/2.10/v/string_type Tool Search a PDF's content accepts these inputs: Tool Name: Search a PDF's content Tool Arguments: {'query': {'description': "Mandatory query you want to use to search the PDF's content", 'type': 'str'}} Tool Description: A tool that can be used to semantic search a query the /content/drive/MyDrive/LLM_crewai_RAG/EngineeringReference.pdf PDF's content. # Agent: Research Agent ## Thought: I need to search for the term "Baffle Heat Balance" in the PDF to find the relevant information that addresses the question. ## Using tool: Search a PDF's content ## Tool Input: "{'query': {'description': 'Baffle Heat Balance', 'type': 'str'}}" ## Tool Output: I encountered an error while trying to use the tool. This was the error: Arguments validation failed: 1 validation error for FixedPDFSearchToolSchema query Input should be a valid string [type=string_type, input_value={'description': 'Baffle H...Balance', 'type': 'str'}, input_type=dict] For further information visit https://errors.pydantic.dev/2.10/v/string_type Tool Search a PDF's content accepts these inputs: Tool Name: Search a PDF's content Tool Arguments: {'query': {'description': "Mandatory query you want to use to search the PDF's content", 'type': 'str'}} Tool Description: A tool that can be used to semantic search a query the /content/drive/MyDrive/LLM_crewai_RAG/EngineeringReference.pdf PDF's content.. Moving on then. I MUST either use a tool (use one at time) OR give my best final answer not both at involves the thermal energy emitted by the baffle and the energy it absorbs from its surroundings. - **q' conv,Env** accounts for the heat loss or gain due to convective heat transfer with the air outside the baffle. Convection refers to the transfer of heat through fluid motion—in this case, the air. - **q' LW R,cav** involves long wavelength	the same time. To Use the following format: Thought: you should always think about what to do Action: the action to take, should be one of [Search a PDF's content] Action Input: the input to the action, dictionary enclosed in curly braces Observation: the result of the action ... (this Thought/Action/Action Input/Result can repeat N times) Thought: I now can give a great answer Final Answer: Your final answer must be the great and the most complete as possible, it must be outcome described # Agent: Research Agent ## Thought: Thought: I will attempt to search again for the term "Baffle Heat Balance" in the PDF to find the relevant information, making sure the input format is correct. ## Using tool: Search a PDF's content ## Tool Input: "{'query': 'Baffle Heat Balance'}" ## Tool Output: Relevant Content: 4.1. EXTERIOR NA TURALL Y VENTED CAVITY 179 Figure 4.1: Baffle Surface Heat Balance heat transfer surfaces should reflect the construction of just the underlying surface. The SurfaceProperty:ExteriorNaturalVentedCavity object is used to describe the detached layer, or baffle, and the characteristics of the cavity and openings for natural ventilation. This model uses the SurfaceProperty:OtherSideConditionsModel object to pass boundary conditions to the heat transfer modeling for the underlying surfaces. 4.1.1 Baffle Heat Balance The baffle is assumed to be sufficiently thin and high-conductivity so that it can be modeled using a single temperature (for both sides and along its area). This temperature Ts,baff is determined by formulating a heat balance on a control volume that just encapsulates the baffle surface. The baffle is assumed to completely cover the underlying surface such that it is opaque to shortwave and longwave radiation. This assumption means that even though the baffle will have some open area for ventilation, no solar energy passes through these openings. The heat balance is diagrammed in the following figure. The heat balance on the baffle surface's control volume is $q'_{\text{sol}} + q'_{\text{LW R,Env}} + q'_{\text{conv,Env}} + q'_{\text{LW R,cav}} + q'_{\text{conv,cav}} + q'_{\text{source}} = 0$ (4.1) where: q' sol is absorbed direct and diffuse solar (short wavelength) radiation heat flux. Q' LW R,Env is net long wavelength (thermal) radiation flux exchange with the air and surroundings. 178 # Agent: Research Agent ## Final Answer: The baffle is assumed to be sufficiently thin and high-conductivity so that it can be modeled using a radiation exchange occurring within any cavities or spaces adjacent to the baffle. - **q' conv,cav** is the convective heat transfer happening within these cavities. - **q' source** includes any additional heat sources present in the vicinity of the baffle. To simplify, this equation allows us to understand	single temperature (for both sides and along its area). This temperature, Ts,baff, is determined by formulating a heat balance on a control volume that just encapsulates the baffle surface. The baffle is assumed to completely cover the underlying surface such that it is opaque to shortwave and longwave radiation. This assumption means that even though the baffle will have some open area for ventilation, no solar energy passes through these openings. The heat balance on the baffle surface's control volume is expressed as: $q'_{\text{sol}} + q'_{\text{LW R,Env}} + q'_{\text{conv,Env}} + q'_{\text{LW R,cav}} + q'_{\text{conv,cav}} + q'_{\text{source}} = 0$ (4.1) Where: - q' sol is absorbed direct and diffuse solar (short wavelength) radiation heat flux. - q' LW R,Env is net long wavelength (thermal) radiation flux exchange with the air and surroundings. # Agent: Professional Re-Writer ## Task: Re-Write improved professional answer based on the research agent's findings, to make sure it is understandable by non-building expert by explaining terminologies # Agent: Professional Re-Writer ## Final Answer: In the context of building physics, a baffle is a thin barrier made of high conductivity material, which allows us to assume that it maintains a uniform temperature across its entire surface. This uniform temperature is designated as Ts,baff. To determine this temperature, we use a concept called "heat balance," which involves analyzing the thermal energy that enters and exits a specified volume surrounding the baffle. For our analysis, we visualize a 'control volume' that encapsulates the baffle. The baffle is considered to cover the underlying surface completely, meaning it does not allow shortwave and longwave radiation to pass through. Essentially, this means that despite having some openings for air circulation, sunlight cannot penetrate through these areas, preventing any direct solar energy from affecting the baffle's temperature. The heat balance equation helps capture all forms of heat energy interacting with the baffle and is expressed mathematically as follows: $q'_{\text{sol}} + q'_{\text{LW R,Env}} + q'_{\text{conv,Env}} + q'_{\text{LW R,cav}} + q'_{\text{conv,cav}} + q'_{\text{source}} = 0$ (4.1) Here's a breakdown of each term in the equation: - **q' sol** refers to the heat flux from both direct sunlight (shortwave radiation) and diffuse solar energy that's absorbed by the baffle. - **q' LW R,Env** represents the net long wavelength radiation exchange between the baffle and the surrounding air and environment. This how the baffle interacts thermally with its environment, ensuring that all incoming and outgoing thermal energy is accounted for and balanced, which ultimately determines its surface temperature.
---	--	--

Listing 9. Step-by-step process of agentic workflow in Case Study 4.

Data availability

Data will be made available on request.

References

- [1] P. MacNaughton, X. Cao, J. Buonocore, J. Cedeno-Laurent, J. Spengler, A. Bernstein, J. Allen, Energy savings, emission reductions, and health co-benefits of the green building movement, *J. Expo. Sci. Environ. Epidemiol.* 28 (4) (2018) 307–318, <https://doi.org/10.1038/s41370-017-0014-9>.
- [2] D.B. Crawley, L.K. Lawrie, F.C. Winkelmann, W.F. Buhl, Y.J. Huang, C.O. Pedersen, R.K. Strand, R.J. Liesen, D.E. Fisher, M.J. Witte, EnergyPlus: creating a new-generation building energy simulation program, *Energ. Build.* 33 (4) (2001) 319–331, [https://doi.org/10.1016/S0378-7788\(00\)00114-6](https://doi.org/10.1016/S0378-7788(00)00114-6).
- [3] X. Li, J. Wen, Review of building energy modeling for control and operation, *Renew. Sust. Energ. Rev.* 37 (2014) 517–537, <https://doi.org/10.1016/j.rser.2014.05.056>.
- [4] C.F. Reinhart, C.C. Davila, Urban building energy modeling—a review of a nascent field, *Build. Environ.* 97 (2016) 196–202, <https://doi.org/10.1016/j.buildenv.2015.12.001>.
- [5] L. Zhang, J. Wen, Y. Li, J. Chen, Y. Ye, Y. Fu, W. Livingood, A review of machine learning in building load prediction, *Appl. Energy* 285 (2021) 116452, <https://doi.org/10.1016/j.apenergy.2021.116452>.
- [6] Z. Deng, Y. Chen, J. Yang, F. Causone, AutoBPS: a tool for urban building energy modeling to support energy efficiency improvement at city-scale, *Energ. Build.* 282 (2023) 112794, <https://doi.org/10.1016/j.enbuild.2023.112794>.
- [7] H.A. Toosi, M. Lavagna, F. Leonforte, C. Del Pero, N. Aste, Life cycle sustainability assessment in building energy retrofitting: A review, *Sustain. Cities Soc.* 60 (2020) 102248, <https://doi.org/10.1016/j.scs.2020.102248>.
- [8] J. Keirstead, M. Jennings, A. Sivakumar, A review of urban energy system models: approaches, challenges and opportunities, *Renew. Sust. Energ. Rev.* 16 (6) (2012) 3847–3866, <https://doi.org/10.1016/j.rser.2012.02.047>.
- [9] L. Aye, T. Ngo, R.H. Crawford, R. Gammampila, P. Mendis, Life cycle greenhouse gas emissions and energy analysis of prefabricated reusable building modules, *Energ. Build.* 47 (2012) 159–168, <https://doi.org/10.1016/j.enbuild.2011.11.049>.
- [10] W. Gu, Z. Wu, R. Bo, W. Liu, G. Zhou, W. Chen, Z. Wu, Modeling, planning and optimal energy management of combined cooling, heating and power microgrid: a review, *Int. J. Electr. Power Energy Syst.* 54 (2014) 26–37, <https://doi.org/10.1016/j.ijepes.2013.06.028>.
- [11] V. Michalakopoulos, S. Pelekis, G. Korpapakis, V. Karakolis, S. Mouzakitis, D. Askounis, Data-driven building energy efficiency prediction using physics-informed neural networks, in: *IEEE Conference on Technologies for Sustainability (SusTech)*, IEEE, 2024, pp. 84–91, <https://doi.org/10.1109/SusTech60925.2024.10553513>.
- [12] J. Lee, S. Yoon, Metadata schema for virtual building models in digital twins: VB schema implemented in GPT-based applications, *Energ. Build.* 327 (2025) 115039, <https://doi.org/10.1016/j.enbuild.2024.115039>.
- [13] J. Song, S. Yoon, Ontology-assisted GPT-based building performance simulation and assessment: implementation of multizone airflow simulation, *Energ. Build.* 325 (2024) 114983, <https://doi.org/10.1016/j.enbuild.2024.114983>.
- [14] K. Amasyali, N.M. El-Gohary, A review of data-driven building energy consumption prediction studies, *Renew. Sust. Energ. Rev.* 81 (2018) 1192–1205, <https://doi.org/10.1016/j.rser.2017.04.095>.
- [15] Y. Chen, M. Guo, Z. Chen, Z. Chen, Y. Ji, Physical energy and data-driven models in building energy prediction: a review, *Energy Rep.* 8 (2022) 2656–2671, <https://doi.org/10.1016/j.egyr.2022.01.162>.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017) 6000–6010, <https://doi.org/10.5555/3295222.3295349>.
- [17] A. Sharma, X. Li, H. Guan, G. Sun, L. Zhang, L. Wang, K. Wu, L. Cao, E. Zhu, A. Sim, Automatic data transformation using large language model—an experimental study on building energy data, in: *IEEE International Conference on Big Data (BigData)*, IEEE, 2023, pp. 1234–1243, <https://doi.org/10.1109/BigData59044.2023.10386931>.
- [18] G. Jiang, Z. Ma, L. Zhang, J. Chen, EPlus-LLM: a large language model-based computing platform for automated building energy modeling, *Appl. Energy* 367 (2024) 123431, <https://doi.org/10.1016/j.apenergy.2024.123431>.
- [19] L. Zhang, Z. Chen, Large language model-based interpretable machine learning control in building energy systems, *Energ. Build.* 313 (2024) 114278, <https://doi.org/10.1016/j.enbuild.2024.114278>.
- [20] S. Khadka, L. Zhang, Scaling Data-Driven Building Energy Modelling using Large Language Models, *arXiv preprint arXiv:2407.03469*, <https://arxiv.org/abs/2407.03469>, 2024.
- [21] K. Forth, A. Borrmann, Semantic enrichment for BIM-based building energy performance simulations using semantic textual similarity and fine-tuning multilingual LLM, *J. Build. Eng.* 95 (2024) 110312, <https://doi.org/10.1016/j.jobe.2024.110312>.
- [22] Y. Fu, H. Peng, A. Sabharwal, P. Clark, T. Khot, Complexity-based prompting for multi-step reasoning, in: *The Eleventh International Conference on Learning Representations*, 2022, <https://doi.org/10.48550/arXiv.2210.00720>.
- [23] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, A survey on large language model based autonomous agents, *Front. Comp. Sci.* 18 (6) (2024) 186345, <https://doi.org/10.1007/s11704-024-40231-1>.
- [24] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, Self-refine: iterative refinement with self-feedback, *Adv. Neural Inf. Process. Syst.* 36 (2024) 12345–12356, <https://doi.org/10.48550/arXiv.2303.17651>.
- [25] S.G. Patil, T. Zhang, X. Wang, J.E. Gonzalez, Gorilla: Large language model connected with massive apis, *arXiv preprint arXiv:2305.15334*, 2023, <https://doi.org/10.48550/arXiv.2305.15334>.
- [26] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q.V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, *Adv. Neural Inf. Process. Syst.* 35 (2022) 24824–24837, <https://doi.org/10.5555/3600270.3602070>.
- [27] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, M. Sun, ChatDev: Communicative agents for software development, *arXiv preprint arXiv:2307.07924*, 2023, p. 6, <https://doi.org/10.48550/arXiv.2307.07924>.
- [28] L. Zhang, V. Ford, Z. Chen, J. Chen, Automatic building energy model development and debugging using large language models agentic workflow, *Energ. Build.* (2024) 115116, <https://doi.org/10.1016/j.enbuild.2024.115116>.
- [29] T. Xiao, P. Xu, Exploring automated energy optimization with unstructured building data: a multi-agent based framework leveraging large language models, *Energ. Build.* 322 (2024) 114691, <https://doi.org/10.1016/j.enbuild.2024.114691>.
- [30] L. Zhang, Z. Chen, V. Ford, Advancing building energy modeling with large language models: exploration and case studies, *Energ. Build.* 323 (2024) 114788, <https://doi.org/10.1016/j.enbuild.2024.114788>.
- [31] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, in: *International Conference on Learning Representations (ICLR)*, 2023, <https://arxiv.org/abs/2210.03629>.
- [32] S. Porsdam Mann, B.D. Earp, N. Möller, S. Vynn, J. Savulescu, AUTOGEN: a personalized large language model for academic enhancement—ethics and proof of principle, *Am. J. Bioeth.* 23 (10) (2023) 28–41, <https://doi.org/10.1080/15265161.2023.2233356>.
- [33] J. Wang, Z. Duan, Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models, *arXiv preprint arXiv:2412.03801*, <https://arxiv.org/abs/2412.03801>, 2024.
- [34] S. Joshi, Advancing innovation in financial stability: a comprehensive review of ai agent frameworks, challenges and applications, *World J. Adv. Eng. Technol. Sci.* 14 (2) (2025) 117–126, <https://doi.org/10.30574/wjaets.2025.14.2.0071>.
- [35] F. Oliveira, L. Gomes, Z. Vale, Retrieval-augmented generation powered by a multi-agent system to assist the operation of industries, in: *International Conference on Practical Applications of Agents and Multi-Agent Systems*, Springer, 2024, pp. 208–216, https://doi.org/10.1007/978-3-031-70415-4_18.
- [36] X. Guan, Z. Wu, LLM Agents: How They Work and Where They Go Wrong, *Holistic AI Blog*, 2025. Available from: <https://www.holisticai.com/blog/llm-agents-use-cases-risks>. (Accessed 22 April 2025).
- [37] B. Ghosh, Building the right LLM agent framework, *Medium* (2024). Available from: <https://medium.com/%40bijit211987/building-the-right-llm-agent-framework-cb82aa6c976c>. (Accessed 22 April 2025).
- [38] L. Zhang, V. Ford, Z. Chen, J. Chen, Automatic building energy model development and debugging using large language models agentic workflow, *Energ. Build.* 327 (2025) 115116, <https://doi.org/10.1016/j.enbuild.2024.115116>.
- [39] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, Retrieval-augmented generation for knowledge-intensive nlp tasks, *Adv. Neural Inf. Process. Syst.* 33 (2020) 9459–9474, <https://doi.org/10.5555/3495724.3496517>.
- [40] M. Deru, K. Field, D. Studer, K. Benne, B. Griffith, P. Torcellini, B. Liu, M. Halverson, D. Winiarski, M. Rosenberg, US Department of Energy commercial reference building models of the national building stock (Technical Report NREL/TP-5500-46861), National Renewable Energy Laboratory (NREL), 2011. <https://www.nrel.gov/docs/fy11osti/46861.pdf>. (Accessed 22 April 2025).